Zephyr Project

IT-Schulungen.com
Technology & Transformation

Callback-Grenze, zbus und Workqueue

Vom GPIO-Callback über event_input_chan zur beobachtbaren Verarbeitung

3.11

Zeitlinie des zbus-Ereignispfads

AUSFÜHRUNGSKONTEXTE

```
Shell → Stimulus → GPIO-Callback → zbus → Async-Listener
```

Shell-Producer	publiziert event_command im Threadkontext
Stimulus-Subscriber	setzt den Pegel über den native_sim-Adapter
GPIO-Callback	erstellt event_input und publiziert mit K_NO_WAIT
zbus Async-Listener	verarbeitet das Event später in der Workqueue
event stats	vergleicht Command-, Input- und Fehlerzähler

Der Publisher-Kontext endet am zbus-Kanal; Fachlogik läuft erst im Async-Listener.

Die Callback-Grenze

01	`Erfassen`	Pegel und Zeitmarke in struct event_input übernehmen
02	`Publizieren`	zbus_chan_pub(..., K_NO_WAIT) auslösen und Rückgabewert sichern
03	`Vermeiden`	Logging, Wartezeiten, Allokation und Fachlogik

Der Callback bleibt ISR-tauglich; zbus transportiert die Nachricht in einen Threadkontext.

zbus-Observer nach Kontext wählen

LISTENER

Im Publisher-Kontext

sofortige Callback-Ausführung

nur für sehr kurze Reaktionen

ungeeignet für Fachlogik

ASYNC-LISTENER

Auf der Workqueue

Callback wird verzögert

Fachlogik läuft im Threadkontext

Wahl für das Tagesprojekt

MESSAGE-SUBSCRIBER

Eigener Thread

Nachrichtenkopie pro Publish

Reihenfolge bleibt erhalten

mehr Ressourcen und Konfiguration

Für event_input_chan nutzt das Tagesprojekt einen Async-Listener; Listener bleibt bewusst außen vor.

Callback: Event publizieren

GPIO-CALLBACK

```
struct event_input evt = {
    .level = (pins & BIT(INPUT_PIN)) != 0,
    .timestamp = k_cycle_get_32(),
};

int rc = zbus_chan_pub(&event_input_chan,
                       &evt, K_NO_WAIT);
if (rc != 0) {
    atomic_inc(&input_publish_failed);
}
```

01 ISR-tauglich

K_NO_WAIT ist Pflicht; keine Shell-Ausgabe und kein blockierender Pfad.

02 Fehler sichtbar

Ein fehlgeschlagener Publish erhöht input_publish_failed.

Der Channel kopiert event_input; die lokale Nachricht darf auf dem Stack liegen.

Async-Listener und Überlast

01	`Verarbeiten`	Async-Listener läuft auf der System-Workqueue und erhöht processed
02	`Begrenzen`	lange oder blockierende Fachlogik verzögert andere Work-Items
03	`Messen`	input_published, processed und input_publish_failed getrennt führen

Für schwere oder blockierende Arbeit ist ein eigener Subscriber-Thread die bessere Erweiterung.

event stats als zbus-Diagnosefläche

Command-Pfad	cmd_received, cmd_rejected und laufende Sequenz
Input-Pfad	input_published, processed und letzter Pegel
Publish-Fehler	input_publish_failed und letzter Rückgabewert
Zeitmarken	letzter Callback und letzte Verarbeitung
Bilanz	Command-, Input- und Verarbeitungszähler vergleichen

Diagnose erfolgt auf Anfrage in der Shell – niemals direkt im GPIO-Callback.

Interaktiv prüfen, automatisiert absichern

INTERAKTIV

zbus-Pfad erkunden

event input, pulse und burst

event stats vergleichen

Publish-Fehler beobachten

native_sim als Primärziel

AUTOMATISIERT

Bilanz beweisen

eine Command-Nachricht

erwartete Input-Events

processed == input_published

Fehlerzähler bleibt null

KONTEXTREVIEW

Ausführung zuordnen

Shell: Producer

Stimulus: Subscriber

GPIO: K_NO_WAIT Publisher

Fachlogik: Async-Listener